

DevSecOps Deployment Procedure

Standard Application Template & Context Document

Document Version: 1.0.0

Date: September 2026

Status: Approved / Production Ready

Target Audience: DevOps Engineers, Security Analysts, Developers

1. Executive Summary & Philosophy

This document details the secure deployment roadmap, pipeline gates, and context architecture for our standard application template. Our DevSecOps methodology centers around '**shifting security left**', embedding robust automated scanning mechanisms as close to the developer environment as possible to mitigate risks before code reaches production environments.

2. Standard Application Template Context

Our standard application utilizes a modern componentized stack optimized for cloud-native orchestration. Understanding this setup guarantees robust CI/CD execution and pinpointing of telemetry hooks.

2.1 Tech Stack Components

Layer	Technology	Scan Target Context
Frontend	React / TypeScript / Tailwind CSS	Source vulnerabilities, cross-site scripting (XSS)
Package Manager	NodeJS / npm (package.json)	Third-party transitive dependency vulnerabilities
Containerization	Docker (Multi-stage builds)	Base image misconfigurations, root-less execution
Orchestration	Kubernetes (AKS/EKS)	Network policies, secrets injection, pod security limits

3. Continuous DevSecOps Deployment Pipeline

The automated lifecycle comprises structural safety gates that block code integration upon violating security baselines.

- **Phase 1: Code Commit & IDE:** Local hooks execute lightweight pre-commit rules ensuring no hardcoded plaintext passwords or keys are pushed into source repositories.
- **Phase 2: Build & Static Scans:** Pipeline provisions a secure runner, downloads isolated dependencies via package locks, and performs Static Application Security Testing (SAST) and Software Composition Analysis (SCA).
- **Phase 3: Image Artifact Quarantine:** Builds a multi-stage Docker image and executes container scanning immediately before promoting images to the private registry.
- **Phase 4: Continuous Delivery (GitOps):** GitOps controllers (e.g., ArgoCD) synchronize hardened manifest definitions straight onto the target Kubernetes cluster.
- **Phase 5: Dynamic Testing & Audit:** Triggers non-intrusive Dynamic Application Security Testing (DAST) on live staging routes, logging runtime behaviors into standard dashboards.

3.1 Sample Pipeline Implementation Context (GitHub Actions)

```
name: DevSecOps Microservice Pipeline
on: [push, pull_request]

jobs:
  security_scan:
    runs-on: ubuntu-latest
    steps:
      - name: Code Checkout
        uses: actions/checkout@v4

      - name: Execute Trivy File System Scan
        run: |
          trivy fs --severity HIGH,CRITICAL --exit-code 1 .

  build_and_publish:
    needs: security_scan
    runs-on: ubuntu-latest
    steps:
      - name: Build Container Image
        run: docker build -t internal-registry/app:${{ github.sha }} .
```

4. Hardened Controls & Risk Mitigation Thresholds

Scan Domain	Automation Tool	Failure / Break Criterion
Static Analysis (SAST)	SonarQube / Semgrep	Any blocker / critical security finding
Dependencies (SCA)	Snyk / OWASP Dependency-Check	CVSS v3 Score ≥ 8.0 with existing fix paths
Secrets Auditing	Gitleaks / Trufflehog	100% strict block on match of high-entropy strings
Container Manifests	Trivy / Clair	Unpatched vulnerability with public exploits active
Dynamic Scanning (DAST)	OWASP ZAP	High severity OWASP Top 10 vulnerabilities active

5. Runtime Protection & Incident Management

Post-deployment operations mandate constant behavioral monitoring. Production namespaces are restricted via cluster admission controllers preventing untrusted registries. Runtime logs route straight to a centralized SIEM, where threshold alerts prompt automated containment steps upon malicious system calls or anomalous file system updates.